



ATTRIBUTE DIRECTIVE IN ANGULAR



codewithrutu_09

✓ What Are Attribute Directives?

Attribute directives allow you to **dynamically change the appearance or behavior** of DOM elements.

Attribute directives are used to **modify the look or behavior** of DOM elements, components, or other directives.

- Change **style, class, or attributes**
- Do **not** add/remove DOM elements
- Manipulate element properties

🔍 Common examples:

`ngClass, ngStyle, [disabled], [hidden]`

✓ Using ngClass and ngStyle

```
app.component.html  
  
<div [ngClass]="{ active: isActive, error: hasError }">  
  Conditional Classes  
</div>
```

```
app.component.html  
  
<p [ngStyle]="{ color: isError ? 'red' : 'green' }">  
  Status Message  
</p>
```

📌 Easy way to bind dynamic styling to your logic

✓ **Why Use Attribute Directives?**

↺ Reuse logic-driven behavior/styling

🎨 Apply consistent UI effects (e.g., highlights, validation)

🔍 Encapsulate presentation behavior

📦 Clean up your templates

Example use cases:

- Highlight invalid fields
- Auto-focus inputs
- Add tooltips or animations

✓ Creating a Custom Attribute Directive



```
ng generate directive highlight
```



app.component.ts

```
@Directive({ selector: '[appHighlight]' })  
export class HighlightDirective {  
  constructor(private el: ElementRef) {  
    el.nativeElement.style.backgroundColor =  
    'yellow';  
  }  
}
```



```
<p appHighlight>I'm highlighted!</p>
```

✓ Add More Power with @HostListener & @HostBinding

```
app.component.ts

@Directive({ selector: '[appHover]' })
export class HoverDirective {
  @HostListener('mouseenter') onMouseEnter() {
    this.el.nativeElement.style.color = 'blue';
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.el.nativeElement.style.color = 'black';
  }

  constructor(private el: ElementRef) {}
}
```

📌 Adds interactivity at the directive level.

Best Practices

- Keep logic focused & reusable
- Prefix with app (e.g., appHighlight) to avoid name clashes
- Inject ElementRef and Renderer2 instead of directly modifying the DOM
- Use HostBinding & HostListener for dynamic styling & event handling
-  Attribute directives change how elements look or behave
-  Create custom ones for reusable logic



codewithrutu_09



Like this post!



Share with your friends



Save it for later



**Tell us what you think in
comment**



codewithrutu_09



Rutuja Shinde